

Plan de desarrollo · Lo que falta

Continúa donde termina `ESTADO-DEL-PRODUCTO.md`, que dice qué hay hoy. Esto dice en qué orden hacer lo que no hay, y por qué en ese orden y no en otro.

Fecha: 18 de agosto de 2026 · ~150 puntos en total

1. Dónde estamos

De las tres promesas del producto, **dos están completas y en producción** —el espacio de trabajo y el control de ventas— y la tercera está empezada: bóveda de credenciales, dos conectores y la Fase 0 del entorno embebido.

Los planes anteriores (`plan-mundo-y-plataforma.md`, `plan-conectores-busqueda-e-interfaz.md`) están cumplidos salvo su último tramo, que es el bloque D de aquí.

2. La idea que ordena este plan

Al contar el producto desde el código aparece un desequilibrio que no se ve usándolo: **está más adelantado como producto que como sistema que hay que sostener**.

Hay cinco usuarios reales, una organización con su historia dentro, grabaciones de llamadas y credenciales de terceros cifradas en la bóveda. Y **no hay ninguna copia de seguridad**. Ni de Postgres, ni del almacén. Si el disco de esa máquina falla esta noche, no se pierde una demo: se pierde el producto entero y las cuentas de todo el equipo.

> **Construir S7 encima de un sistema sin copias de seguridad es levantar otro > piso sobre unos cimientos que nadie ha mirado.**

Por eso el bloque A va primero aunque no sea una función y no se vea en pantalla. Son además **los puntos más baratos de todo el plan**: horas, no semanas, y son los únicos que protegen lo que ya existe en vez de añadir algo encima.

La segunda idea que ordena esto: **lo que está construido y sin fusionar es deuda, no progreso**. Hay una rama con el arreglo del 415 hecho en el servidor y cinco pruebas de navegador en Playwright, escritas y paradas. Cuesta más mantenerlas vivas fuera que meterlas.

3. Bloque A — Que nada se pierda · ~14 puntos

Nada de esto añade una función. Todo esto evita perder lo que ya hay.

3.1 Copias de seguridad de Postgres y del almacén · ~8 pts

Un servicio en el compose que haga `pg_dump` a diario y suba el resultado fuera de esa máquina, más lo mismo para el bucket de MinIO. Con retención y, sobre todo, **con una restauración probada**: una copia que nadie ha restaurado nunca no es una copia, es una carpeta con archivos grandes.

Lo primero que hay que decidir es **dónde** van, porque una copia en el mismo disco no protege del caso que importa.

3.2 Registrar el runner autoalojado · ~1 pt

`deploy.yml` ya está en el repositorio y hace lo correcto: espera a que la integración continua termine en verde y despliega. Pero el runner está descargado en `C:\Users\Juan\actions-runner` **sin configurar** — no hay `.runner`, ni credenciales, ni servicio—, así que los despliegues se quedan en cola.

Es un `config.cmd` con un token del repositorio y un `svc.cmd install`. Lo tiene que hacer una persona porque el token es suyo.

Cuidado con una consecuencia: el despliegue construye desde `C:\Users\Juan\DevUP`, la misma carpeta donde se trabaja. Con el runner activo, una edición a medio hacer se va a producción aunque no esté commiteada. O se asume, o el despliegue pasa a clonar aparte y hay que resolver dónde vive `.env.production`.

3.3 SMTP real · ~3 pts

Hoy solo está puesto `MAIL_FROM`; no hay servidor. Las invitaciones y los correos de recuperación **se escriben en el registro del servidor** en vez de enviarse. Funciona para probar y no funciona para usar: nadie ajeno al equipo puede entrar por su cuenta ni recuperar su contraseña.

3.4 Custodia de `VAULT_MASTER_KEY` · ~2 pts

Descifra todas las credenciales de terceros guardadas. Si se pierde o cambia, todas las conexiones quedan indecifrables para siempre. Hoy vive en un único `.env.production` en una sola máquina, sin copia y sin procedimiento escrito de rotación. Escribir ese procedimiento —y guardar la clave en otro sitio— es más barato que descubrir el problema el día que haga falta.

4. Bloque B — Cerrar lo que quedó a medias · ~13 puntos

Cosas empezadas que hoy cuestan más abiertas que cerradas.

4.1 Fusionar el arreglo del 415 hecho en el servidor · ~2 pts

En `claude/inicio-desarrollo-nu1ftu` hay un analizador comodín que acepta cuerpo vacío venga el `Content-Type` que venga, con prueba de regresión. Lo que está en main resuelve el mismo fallo **en el cliente**, que solo protege a las llamadas que pasan por nuestro `api.ts`.

Revisar, fusionar, y quitar el arreglo del cliente si el del servidor lo hace redundante.

4.2 TURN · ~5 pts

`STUN_URLS` está puesto; `TURN_URLS`, `TURN_SECRET` y `TURN_STATIC_USERNAME` están **vacíos**. La API ya lo detecta y avisa a la interfaz, así que el trabajo no es de código: es levantar o contratar un TURN y rellenar esas variables.

Sin él la llamada conecta y parece correcta, pero en NAT simétrico —buena parte de las redes móviles— no llega el audio. Es el tipo de fallo que se descubre a mitad de una reunión con alguien de fuera.

4.3 Sacar Spotify del modo desarrollo · ~1 pt

Dos límites que **no se arreglan con código**: solo 5 cuentas además del dueño pueden usar la aplicación, y las canciones de una lista no se pueden leer. Los dos se levantan pidiendo la extensión de cuota en el panel de Spotify. Mientras tanto, añadir a los compañeros en *User Management* es lo que desbloquea a cada uno.

4.4 Explicar los 404 del conector de GitHub · ~3 pts

Un 404 al añadir un repositorio casi siempre es que el token de alcance `repo` no lo tiene autorizado, o que la organización bloquea esos tokens sin aprobar. No es un fallo de DevUP, pero la interfaz no lo dice y parece uno. Es texto y una comprobación, no arquitectura.

4.5 Cerrar las decisiones 0003 y 0004 · ~2 pts

Siguen como propuestas: arquitectura de despliegue (monolito frente a microservicios, VPS con y sin coste) y el conector de GitHub embebido con la semilla de agente. La 0003 condiciona el bloque D y la 0004 condiciona el F, así que decidir las es requisito de los dos.

5. Bloque C — Pruebas que abren un navegador · ~10 puntos

`ci.yml` ya cubre lo difícil en cada push: tipos, migraciones, **aislamiento entre organizaciones**, sala del mundo, migraciones idempotentes y build. Eso es más de lo que tiene la mayoría de proyectos a esta altura.

Lo que **no** cubre es nada que abra un navegador. Los 13 guiones de `e2e/` se lanzan a mano, y ahí es donde viven los fallos que ninguna prueba de tipos puede ver: el bucket del almacén que nunca se creaba pasó meses sin detectarse porque solo se manifestaba a mitad de una subida real.

El trabajo: traer las 5 pruebas de Playwright de `claude/inicio-desarrollo-nu1ftu`, decidir si los 13 guiones `.mjs` se migran o conviven, y añadir un job al workflow que levante la pila y las corra.

La trampa: una prueba de navegador que falla a veces es peor que ninguna, porque enseña al equipo a ignorar el rojo. Mejor cinco estables que veinte inestables.

6. Bloque D — S7: vista unificada de infraestructura · ~22 puntos

Es lo que cierra la tercera promesa, y lo que más se ve.

Entornos y despliegues del cliente en una sola pantalla, sobre la bóveda que ya existe. Vale doble por un motivo que no es evidente: **es lo que enciende los muebles vivos que están puestos en DevVerse y no hacen nada** —la pantalla de despliegue y el rack de servidores—. Hoy son decorado; con esto pasan a mostrar el estado real.

Depende de la decisión 0003, porque el modelo de despliegue decide qué se puede enseñar.

7. Bloque E — Base de datos como código · ~20 puntos

Migraciones del cliente sincronizadas con su repositorio, con el mismo criterio que ya se aplica aquí: append-only, idempotentes, y con la política de aislamiento como parte de la migración y no como un paso aparte.

Aquí hay una ventaja injusta que conviene aprovechar: el proyecto ya tiene 20 migraciones y una regla dura —**toda tabla nueva necesita política y un caso en** `isolation.test.ts` — aprendida a base de un fallo silencioso que costó la migración 0018. Ese criterio es el producto, no un detalle interno.

8. Bloque F — Agentes · ~30 puntos

Codex y Claude Code sobre el entorno embebido que ya arranca en `/dev`.

Es el bloque más grande y el más incierto, y depende de la decisión 0004. Antes de escribir código hay dos cosas que decidir: **qué puede tocar un agente** (¿abre PRs?, ¿escribe en la rama?, ¿ejecuta?) y **con qué credenciales** —que es otra vez la bóveda, y por eso conviene que llegue después de que S7 la haya puesto a prueba con algo más que dos conectores.

9. Bloque G — DevUP ID y beta · ~25 puntos

Identidad propia y la apertura a gente de fuera del equipo.

No se puede empezar sin el bloque A. Abrir a usuarios ajenos un sistema sin copias de seguridad y sin correo real no es una beta: es pedirle a alguien que se registre por un enlace que no le llega, para guardar su trabajo en un disco que nadie respalda.

10. Bloque H — Escalar, cuando toque · ~16 puntos

Nada de esto hace falta hoy, y hacerlo antes de tiempo es trabajo tirado. Va escrito para que cuando duela se sepa dónde mirar.

- **Redis para presencia y límite de peticiones** (~8 pts). Hoy ambos viven en

memoria; el día que haya una segunda instancia de API dejarán de ser ciertos.

- **Histéresis de tres radios dentro de una sala** (~8 pts). El reparto por zonas

resuelve veinte personas en cuatro salas; no resuelve doce en una, donde la malla vuelve a pedir once conexiones por cabeza. Descrito en §11 bis de la decisión 0002.

11. El orden, de un vistazo

	BLOQUE	PUNTOS	POR QUÉ AHÍ
1	A · Que nada se pierda	~14	Protege lo que ya existe. Es lo más barato del plan
2	B · Cerrar lo a medias	~13	Deuda que cuesta más abierta que cerrada
3	C · Pruebas de navegador	~10	Antes de añadir superficie, poder comprobarla
4	D · S7 infraestructura	~22	Cierra la tercera promesa y enciende DevVerse
5	E · Base de datos como código	~20	Se apoya en el criterio que ya existe
6	F · Agentes	~30	Necesita la bóveda probada por S7
7	G · DevUP ID y beta	~25	Necesita A entero: copias y correo
8	H · Escalar	~16	Solo cuando duela

12. Riesgos, y lo que no se toca

RIESGO	CÓMO SE EVITA
Una tabla nueva sin política de aislamiento	RLS falla en silencio: afecta 0 filas y sigue. Toda tabla nueva necesita política y un caso en <code>isolation.test.ts</code> , y todo UPDATE que importe comprueba <code>rowCount</code> , no que no haya excepción
<code>VAULT_MASTER_KEY</code> cambia o se pierde	Bloque A.4 antes de que la bóveda guarde nada más
El despliegue automático sube trabajo a medias	Bloque A.2: o se asume, o el runner clona aparte
La oficina inmersiva se toca sin querer	Ningún bloque entra en <code>components/world/</code> , <code>lib/world/</code> , las rutas de <code>devverse</code> ni <code>lib/view-mode.tsx</code>
Pruebas de navegador inestables	Bloque C: cinco estables antes que veinte que fallan a veces

13. Lo que necesito que decidas

- **Copias de seguridad: ¿dónde?** Una copia en el mismo disco no protege del

caso que importa. ¿Otro servidor, un bucket externo, un disco físico? Es la única decisión que bloquea el bloque A entero. 2. **El despliegue automático y la carpeta de trabajo.** ¿Se asume que una edición sin commitear puede irse a producción, o el runner pasa a clonar aparte y movemos `.env.production`? 3. **TURN: ¿propio o contratado?** Levantar un coturn es más trabajo y ningún coste recurrente; contratarlo es al revés. 4. **Spotify: ¿se pide la extensión de cuota?** Sin ella el techo son 5 cuentas y no se pueden listar canciones, y eso condiciona cómo se enseña en una demo. 5. **Decisiones 0003 y 0004.** Bloquean los bloques D y F. Son las dos que hay que cerrar antes de que empiece lo grande. 6. **El orden.** Propongo operación antes que funciones. Si hay una demo con fecha, puede tener sentido adelantar el bloque D — pero conviene que sea una decisión y no un descuido.